How does Spring boot work internally ?
OR

What exctaly happens when you run a spring boot app ?

Spring Boot looks at
1. Frameworks available on the CLASSPATH
(eg : web MVC , Spring Data JPA : ORM --default provider Hibernate, Jackson , DBCP : Hikari)
2. Existing configuration for the application.
Based on these, Spring Boot provides basic configuration needed to configure the application with these frameworks. This is called Auto Configuration.

It uses Spring Boot Starters :
Starters are a set of convenient dependency descriptors that you can include in your application's pom.xml
.
eg : Suppose you want to develop a web application.

W/o Spring boot , we would need to identify the frameworks we want to use, which versions of frameworks to use and how to connect them together.

BUT all web application have similar needs.
These include Spring MVC, Jackson Databind (for data binding), Hibernate-Validator (for server side validation using Java Validation API) and Log4j (for logging). Earlier while creating any web app, we had to choose the compatible versions of all these frameworks.

With Spring boot : You just add Spring Boot Starter Web.

Dependency for Spring Boot Starter Web
```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Just by adding above starter , it will add lot of JARs under maven dependencies

Another eg : If you want to use Spring and JPA for database access, just include the spring-boot-starter-data-jpa dependency in your project, and you are good to go.

Important components of a Spring Boot Application

1. Below is the starting point of a Spring Boot Application

```
@SpringBootApplication
public class HelloSpringBootApplication {

 public static void main(String[] args) {
  SpringApplication.run(HelloSpringBootApplication.class, args);
 }

}
```
About : org.springframework.boot.SpringApplication
It's a Class used to bootstrap and launch a Spring application from a Java main method.

By default class will perform the following steps to bootstrap the application
0.It launches an embedded Tomcat container.
1. Create an ApplicationContext instance (sub i/f : WebApplicationContext --> implementation class --> AnnotationConfigWebApplicationContext : representing SC suitable in web scenario) --meaning launches SC from within Spring boot app.
2. SC manages life cycle of spring beans

@SpringBootApplication - This is where all the spring boot magic happens.
It consists of following 3 annotations.

1. @SpringBootConfiguration
It tells spring boot that this class is a configuration class which can have several bean definitions. We can define various spring beans here and those beans will be available at run time .
(using @Bean annotations)
eg : ModelMapper, PasswordEncoder.AuthenticationManager...

2. @EnableAutoConfiguration
It tells spring boot to automatically configure the spring application based on the dependencies that it sees on the classpath.

eg:
If we have a MySql dependency in our pom.xml , Spring Boot will automatically create a data source,using the properties in application.properties file.

If we have spring web starter , in pom.xml , then spring boot will automatically create the dispatcher servlet n other beans (HandlerMapping , ViewResolver, DispatcherServlet)

All the xml, all the java based configuration is now gone.It all comes for free thanks to spring boot to enable auto configuration annotation.

3. @ComponentScan (equivalent to xml tag : context:component-scan)
with default value of base-packge : supplied at the spring boot configuration time.

For scanning entities : (equivalent to packagesToScan)
@EntityScan(basePackages = "com.app")

So this tells us that spring boot to scan through the classes and see which all classes are marked with the stereotype annotations like @Component Or @Service @Repository and manage these spring beans . Default base-pkg is the pkg in which main class is defined.
Can be overridden by
eg :
@ComponentScan(basePackages = "org.example")
@EntityScan(basePackages ="org.example.model")